

1.0 Number Systems

A **number system** is a method of representing numbers using digits or symbols. It is used in computers to store and process data.

Types of Number Systems:

- **Binary (Base 2):** Uses 0 and 1.
- **Octal (Base 8):** Uses digits 0–7.
- **Decimal (Base 10):** Uses digits 0–9.
- **Hexadecimal (Base 16):** Uses digits 0–9 and A–F.

Importance:

- Binary is the main language of computers.
- Octal and hexadecimal make binary numbers easier to read and write.
- Decimal is used in everyday calculations.

1.1 List the Various Number Systems Used in Digital Computers

The number systems used in digital computers are:

1. **Binary Number System (Base 2)** – Uses digits **0 and 1**.
2. **Octal Number System (Base 8)** – Uses digits **0 to 7**.
3. **Decimal Number System (Base 10)** – Uses digits **0 to 9**.
4. **Hexadecimal Number System (Base 16)** – Uses digits **0 to 9 and A to F**.

These are the four main number systems used in digital computers.

1.2 Explain Decimal Number System

The **Decimal Number System** is the most commonly used number system in daily life. It has a **base (radix) of 10** and uses **10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9**.

In the decimal system, the value of each digit depends on its **position (place value)**. Each position represents a power of 10.

Example:

$$4582 = (4 \times 10^3) + (5 \times 10^2) + (8 \times 10^1) + (2 \times 10^0)$$

Features:

- Base (Radix): **10**

- Digits Used: **0–9**
- Positional value is based on powers of 10.
- Used for everyday calculations.

1.3 Explain Binary Number System

The **Binary Number System** is the number system used by computers. It has a **base (radix) of 2** and uses only **two digits: 0 and 1**.

In the binary system, the value of each digit depends on its **position (place value)**. Each position represents a power of 2.

Example:

$$1011_2 = (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) = 11_{10}$$

Features:

- Base (Radix): **2**
- Digits Used: **0 and 1**
- Positional value is based on powers of 2.
- It is the fundamental number system used in digital computers.

1.4 Explain Octal Number System

The **Octal Number System** is a number system with a **base (radix) of 8**. It uses **8 digits: 0, 1, 2, 3, 4, 5, 6, and 7**.

In the octal system, the value of each digit depends on its **position (place value)**. Each position represents a power of 8.

Example:

$$345_8 = (3 \times 8^2) + (4 \times 8^1) + (5 \times 8^0) = 229_{10}$$

Features:

- **Base (Radix): 8**
- **Digits Used: 0–7**
- **Positional value** is based on powers of 8.
- Used as a shorter representation of binary numbers in digital systems.

1.5 Explain Hexadecimal Number System

The **Hexadecimal Number System** is a number system with a **base (radix) of 16**. It uses **16 symbols: 0–9 and A–F**, where **A = 10, B = 11, C = 12, D = 13, E = 14, and F = 15**.

In the hexadecimal system, the value of each digit depends on its **position (place value)**. Each position represents a power of 16.

Example:

$$2AF_{16} = (2 \times 16^2) + (10 \times 16^1) + (15 \times 16^0) = 687_{10}$$

Features:

- **Base (Radix):** 16
- **Digits Used:** 0–9 and A–F
- **Positional value** is based on powers of 16.
- Used to represent binary data in a shorter and more readable form.

1.6 Convert Decimal Number to Other Base Conversion

A **decimal number** can be converted into binary, octal, and hexadecimal by **repeatedly dividing the number by the required base** and writing the remainders in reverse order.

1.6.1 Decimal to Binary (Base 2)

- Divide the decimal number by **2**.
- Record the remainder.
- Continue dividing the quotient by 2 until the quotient becomes 0.
- Write the remainders **from bottom to top**.

Example: Convert 25_{10} to binary.

$$25 \div 2 = 12, R = 1$$

$$12 \div 2 = 6, R = 0$$

$$6 \div 2 = 3, R = 0$$

$$3 \div 2 = 1, R = 1$$

$$1 \div 2 = 0, R = 1$$

Answer: $25_{10} = 11001_2$

1.6.2 Decimal to Octal (Base 8)

- Divide the decimal number by **8**.
- Record the remainder.
- Continue until the quotient becomes 0.
- Write the remainders **from bottom to top**.

Example: Convert 125_{10} to octal.

$$125 \div 8 = 15, R = 5$$

$$15 \div 8 = 1, R = 7$$

$$1 \div 8 = 0, R = 1$$

Answer: $125_{10} = 175_8$

1.6.3 Decimal to Hexadecimal (Base 16)

- Divide the decimal number by **16**.
- Record the remainder.
- Continue until the quotient becomes 0.
- Write the remainders **from bottom to top**.
- Use **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**.

Example: Convert 254_{10} to hexadecimal.

$$254 \div 16 = 15, R = 14 \text{ (E)}$$

$$15 \div 16 = 0, R = 15 \text{ (F)}$$

Answer: $254_{10} = FE_{16}$

1.7 Convert Binary Number to Other Base Conversion

A **binary number** can be converted into decimal, octal, and hexadecimal using the following methods.

1.7.1 Binary to Decimal

- Multiply each binary digit by its corresponding **power of 2**.
- Add all the results.

Example: Convert 1011_2 to decimal.

$$\begin{aligned} 1011_2 &= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\ &= 8 + 0 + 2 + 1 = 11_{10} \end{aligned}$$

Answer: $1011_2 = 11_{10}$

1.7.2 Binary to Octal

- Group the binary digits into **3 bits** from right to left.
- Convert each group into its octal equivalent.

Example: Convert 110101_2 to octal.

$110\ 101 \rightarrow 6\ 5$

Answer: $110101_2 = 65_8$

1.7.3 Binary to Hexadecimal

- Group the binary digits into **4 bits** from right to left.
- Convert each group into its hexadecimal equivalent.

Example: Convert 10101111_2 to hexadecimal.

$1010\ 1111 \rightarrow A\ F$

Answer: $10101111_2 = AF_{16}$

1.8 Convert Octal Number to Other Base Conversion

An **octal number** can be converted into decimal, binary, and hexadecimal using the following methods.

1.8.1 Octal to Decimal

- Multiply each octal digit by its corresponding **power of 8**.
- Add all the results.

Example: Convert 157_8 to decimal.

$$\begin{aligned} 157_8 &= (1 \times 8^2) + (5 \times 8^1) + (7 \times 8^0) \\ &= 64 + 40 + 7 = 111_{10} \end{aligned}$$

Answer: $157_8 = 111_{10}$

1.8.2 Octal to Binary

- Replace each octal digit with its **3-bit binary equivalent**.

Example: Convert 57_8 to binary.

$5 \rightarrow 101$

$7 \rightarrow 111$

Answer: $57_8 = 101111_2$

1.8.3 Octal to Hexadecimal

- First convert the octal number to **binary**.
- Then group the binary digits into **4 bits** from right to left.
- Convert each group into its hexadecimal equivalent.

Example: Convert 57_8 to hexadecimal.

$57_8 \rightarrow 101111_2 \rightarrow 0010\ 1111 \rightarrow 2F_{16}$

Answer: $57_8 = 2F_{16}$

1.9 Convert Hexadecimal Number to Other Base

Conversion

A **hexadecimal number** can be converted into decimal, binary, and octal using the following methods.

1.9.1 Hexadecimal to Decimal

- Multiply each hexadecimal digit by its corresponding **power of 16**.
- Replace **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**.
- Add all the results.

Example: Convert $2A_{16}$ to decimal.

$$\begin{aligned} 2A_{16} &= (2 \times 16^1) + (10 \times 16^0) \\ &= 32 + 10 = 42_{10} \end{aligned}$$

Answer: $2A_{16} = 42_{10}$

1.9.2 Hexadecimal to Binary

- Replace each hexadecimal digit with its **4-bit binary equivalent**.

Example: Convert AF_{16} to binary.

$A \rightarrow 1010$

$F \rightarrow 1111$

Answer: $AF_{16} = 10101111_2$

1.9.3 Hexadecimal to Octal

- First convert the hexadecimal number to **binary**.
- Then group the binary digits into **3 bits** from right to left.
- Convert each group into its octal equivalent.

Example: Convert $2F_{16}$ to octal.

$2F_{16} \rightarrow 0010\ 1111_2 \rightarrow 000\ 101\ 111 \rightarrow 057_8$

Answer: $2F_{16} = 57_8$

1.10 Binary Number Representation

1.10.1 Define Binary Number Representation

Binary number representation is the method of representing numbers using **binary digits (0 and 1)**. It is used in digital computers to store and process data.

1.10.2 List the Types of Binary Number Representation

The main types of binary number representation are:

1. **Unsigned Binary Number Representation**
 2. **Signed Binary Number Representation**
-

1.10.3 Explain Unsigned Binary Number Representation

In **unsigned binary representation**, all bits are used to represent the magnitude (value) of the number. It can represent **only positive numbers and zero**.

Example (4-bit):

- $0101_2 = 5_{10}$
- $1111_2 = 15_{10}$

Features:

- No sign bit.
 - Represents only positive numbers and zero.
 - All bits store the number value.
-

1.10.4 Explain Signed Binary Number Representation

In **signed binary representation**, the **leftmost bit (Most Significant Bit - MSB)** is used as the **sign bit**.

- **0** → Positive number
- **1** → Negative number

The remaining bits represent the magnitude of the number.

Example (4-bit):

- $0101_2 = +5_{10}$
- $1101_2 = -5_{10}$ (sign-magnitude representation)

Features:

- MSB represents the sign.
- Can represent both positive and negative numbers.
- Commonly used in digital computers for signed values.

1.11 Signed Binary Arithmetic

Signed binary arithmetic is used to perform **addition, subtraction, multiplication, and division** on binary numbers.

1.11.1 Illustrate Addition of Two Signed Binary Numbers

In signed binary addition, numbers with the **same sign are added**, and the sign remains the same.

Example:

```
  0101    (+5)
+ 0011    (+3)
-----
  1000    (+8)
```

Result: $+5 + (+3) = +8$

1.11.2 Illustrate Subtraction of Two Signed Binary Numbers

Subtraction is performed by subtracting one binary number from another.

Example:

```
  1001    (9)
- 0011    (3)
-----
  0110    (6)
```

Result: $9 - 3 = 6$

1.11.3 Illustrate Binary Multiplication

Binary multiplication follows the same method as decimal multiplication.

Rules:

- $0 \times 0 = 0$
- $0 \times 1 = 0$
- $1 \times 0 = 0$
- $1 \times 1 = 1$

Example:

101

$$\begin{array}{r}
 \times \quad 11 \\
 \hline
 101 \\
 + 1010 \\
 \hline
 1111
 \end{array}$$

Result: $101_2 \times 11_2 = 1111_2$

1.11.4 Illustrate Binary Division

Binary division is similar to decimal long division.

Example:

$$1100 \div 10 = 110$$

Verification:

- $1100_2 = 12_{10}$
- $10_2 = 2_{10}$
- $110_2 = 6_{10}$

Result: $1100_2 \div 10_2 = 110_2$

1.12 Binary Coded Decimal (BCD) Coding Scheme

1.12.1 Define Binary Coded Decimal (BCD) Coding Scheme

Binary Coded Decimal (BCD) is a coding scheme in which each decimal digit (0–9) is represented separately by a **4-bit binary code**. It is widely used in digital systems such as calculators and digital clocks.

1.12.2 List the Types of Binary Coded Decimal (BCD)

The main types of BCD codes are:

1. **8421 Code**
2. **2421 Code**
3. **84-2-1 Code**
4. **Excess-3 (XS-3) Code**

1.12.3 Draw and Explain 8421 Code

The **8421 code** is a **weighted BCD code** with weights **8, 4, 2, and 1**.

Decimal 8421 Code

0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Explanation:

Each decimal digit is represented by its equivalent 4-bit binary value.

1.12.4 Draw and Explain 2421 Code

The **2421 code** is a **weighted, self-complementing BCD code** with weights **2, 4, 2, and 1**.

Decimal 2421 Code

0	0000
1	0001

Decimal 2421 Code

2	0010
3	0011
4	0100
5	1011
6	1100
7	1101
8	1110
9	1111

Explanation:

It is called **self-complementing** because the 9's complement of a decimal digit is obtained by complementing its code.

1.12.5 Draw and Explain 84-2-1 Code

The **84-2-1 code** is a **weighted BCD code** with weights **8, 4, -2, and -1**.

Decimal 84-2-1 Code

0	0000
1	0111
2	0110
3	0101
4	0100
5	1011
6	1010

Decimal 84-2-1 Code

7	1001
8	1000
9	1111

Explanation:

This code uses both positive and negative weights and is also a **self-complementing code**.

1.12.6 Draw and Explain Excess-3 Code

The **Excess-3 (XS-3) code** is a **non-weighted, self-complementing code**. It is obtained by **adding 3** to each decimal digit and then converting the result into a 4-bit binary number.

Decimal Excess-3 Code

0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100

Explanation:

Each decimal digit is first increased by **3**, then represented in binary. Excess-3 is commonly used in digital arithmetic because it is self-complementing.

1.13 Character Representation

1.13.1 List Character Representation Codes

The common character representation codes are:

1. **ASCII (American Standard Code for Information Interchange)**
 2. **EBCDIC (Extended Binary Coded Decimal Interchange Code)**
 3. **Unicode**
-

1.13.2 Explain the ASCII Coding Scheme

ASCII (American Standard Code for Information Interchange) is a standard character encoding scheme used to represent letters, numbers, and special symbols in computers.

Features:

- Uses **7 bits** to represent **128 characters** (0–127).
- Extended ASCII uses **8 bits** to represent **256 characters**.
- Includes alphabets (A–Z, a–z), digits (0–9), punctuation marks, and control characters.
- Widely used in computers and communication systems.

Example:

- A = **65** (1000001_2)
 - B = **66** (1000010_2)
 - 0 = **48** (0110000_2)
-

1.13.3 Explain the EBCDIC Coding Scheme

EBCDIC (Extended Binary Coded Decimal Interchange Code) is an **8-bit character encoding scheme** developed by **IBM** for its mainframe and midrange computers.

Features:

- Uses **8 bits** to represent **256 characters**.
- Supports alphabets, numbers, symbols, and control characters.
- Mainly used in **IBM mainframe systems**.
- Less common than ASCII in modern computers.

Example:

- A = **C1** (Hexadecimal)
- B = **C2** (Hexadecimal)
- 0 = **F0** (Hexadecimal)

2.0 Boolean Algebra, Logic Gates and Basic Combinational Circuits

Boolean Algebra is a branch of mathematics used to analyze and design digital logic circuits. It deals with **binary values (0 and 1)** and logical operations such as **AND, OR, and NOT**.

Logic Gates are electronic circuits that perform logical operations on one or more binary inputs to produce a single binary output. The basic logic gates are **AND, OR, NOT, NAND, NOR, XOR, and XNOR**.

Basic Combinational Circuits are digital circuits whose output depends only on the present input values. They do not have memory. Examples include **Half Adder, Full Adder, Half Subtractor, Full Subtractor, Multiplexer (MUX), Demultiplexer (DEMUX), Encoder, and Decoder**.

Applications:

- Digital computers
- Calculators
- Communication systems
- Control systems
- Digital electronic devices

2.1 Boolean Algebra

2.1.1 Define Boolean Algebra

Boolean Algebra is a branch of mathematics used to perform logical operations on binary values **0 and 1**. It is the basis for designing digital circuits.

2.1.2 Explain AND, OR, NOT Operations with Truth Tables

AND Gate ($\cdot$) – Output is **1** only when both inputs are 1.

A B A·B

0 0 0

0 1 0

A B A·B

1 0 0

1 1 1

OR Gate (+) – Output is **1** if any one input is 1.

A B A+B

0 0 0

0 1 1

1 0 1

1 1 1

NOT Gate (') – Output is the complement of the input.

A A'

0 1

1 0

2.1.3 Explain EX-OR and EX-NOR Operations with Truth Tables

EX-OR (XOR) – Output is **1** when the inputs are different.

A B A ⊕ B

0 0 0

0 1 1

1 0 1

1 1 0

EX-NOR (XNOR) – Output is **1** when the inputs are the same.

A B A $\odot$ B

0 0 1

0 1 0

1 0 0

1 1 1

2.1.4 List the Different Postulates in Boolean Algebra

1. Identity Law
2. Null Law
3. Idempotent Law
4. Complement Law
5. Commutative Law
6. Associative Law
7. Distributive Law
8. Absorption Law

2.1.5 State De-Morgan's Theorems

1. $(A + B)' = A' \cdot B'$
2. $(A \cdot B)' = A' + B'$

2.1.6 Prove De-Morgan's Theorems Using Truth Tables

$$(A + B)' = A' \cdot B'$$

A B (A+B)' A'·B'

0 0 1 1

0 1 0 0

$$A \ B \ (A+B)' \ A' \cdot B'$$

$$1 \ 0 \ 0 \quad 0$$

$$1 \ 1 \ 0 \quad 0$$

Both columns are equal. Hence proved.

2.1.7 Simplify Boolean Expressions Using De-Morgan's Theorems

Example:

$$(A + B)' = A'B'$$

$$(AB)' = A' + B'$$

These rules simplify Boolean expressions.

2.1.8 Generate Boolean Expression for a Given Truth Table

Write the Boolean expression by selecting the rows where the output is **1** and combine them using **OR** operation.

2.1.9 Using Sum-of-Products (SOP) Method

- SOP is the **OR** of several **AND** terms (minterms).
- It is obtained from the rows where the output is **1**.

Example:

$$F = A'B + AB'$$

2.1.10 Using Product-of-Sums (POS) Method

- POS is the **AND** of several **OR** terms (maxterms).
- It is obtained from the rows where the output is **0**.

Example:

$$F = (A + B)(A' + B')$$

2.1.11 Use K-Map to Simplify Boolean Expression (Up to 4 Variables)

A **Karnaugh Map (K-Map)** is a graphical method used to simplify Boolean expressions by grouping adjacent 1s (for SOP) or 0s (for POS).

2.1.12 Using Two Variable K-Map

A **2-variable K-Map** contains **4 cells** and is used to simplify expressions with **2 variables (A, B)**.

2.1.13 Using Three Variable K-Map

A **3-variable K-Map** contains **8 cells** and is used to simplify expressions with **3 variables (A, B, C)**.

2.1.14 Using Four Variable K-Map

A **4-variable K-Map** contains **16 cells** and is used to simplify expressions with **4 variables (A, B, C, D)**. It provides the simplest Boolean expression by grouping adjacent cells.

2.2 Logic Gates

2.2.1 Define Logic Gate

A **logic gate** is an electronic circuit that performs a logical operation on one or more binary inputs to produce a single binary output.

2.2.2 List Basic Gates

The basic logic gates are:

1. AND Gate
2. OR Gate
3. NOT Gate

Other gates:

- NAND Gate
 - NOR Gate
 - EX-OR (XOR) Gate
 - EX-NOR (XNOR) Gate
-

2.2.3 Define OR Gate

An **OR gate** is a logic gate whose output is **1** if **any one or both inputs are 1**.

2.2.4 Explain OR Gate with Logic Symbol and Truth Table

Logic Symbol: OR (+)

$$A \text{ OR } B = A + B$$

0 0 0

0 1 1

1 0 1

1 1 1

2.2.5 Define AND Gate

An **AND gate** is a logic gate whose output is **1 only when all inputs are 1**.

2.2.6 Explain AND Gate with Logic Symbol and Truth Table

Logic Symbol: AND ($\cdot$)

$$A \cdot B = Y$$

0 0 0

0 1 0

1 0 0

1 1 1

2.2.7 Define NOT Gate

A **NOT gate** is a logic gate that produces the **complement (inverse)** of the input.

2.2.8 Explain NOT Gate with Logic Symbol and Truth Table

Logic Symbol: NOT (')

$$A = Y'$$

0 1

1 0

2.2.9 What is Universal Gate? List Universal Gates

A **universal gate** is a logic gate that can be used to implement **any Boolean function**.

Universal Gates:

1. NAND Gate
 2. NOR Gate
-

2.2.10 Define NOR Gate

A **NOR gate** is the complement of the OR gate. It gives **1 only when all inputs are 0**.

2.2.11 Explain NOR Gate with Logic Symbol and Truth Table

Output: $Y = (A + B)'$

A	B	Y
---	---	---

0	0	1
---	---	---

0	1	0
---	---	---

1	0	0
---	---	---

1	1	0
---	---	---

2.2.12 Define NAND Gate

A **NAND gate** is the complement of the AND gate. It gives **0 only when all inputs are 1**.

2.2.13 Explain NAND Gate with Logic Symbol and Truth Table

Output: $Y = (A \cdot B)'$

A	B	Y
---	---	---

0	0	1
---	---	---

0	1	1
---	---	---

1	0	1
---	---	---

1	1	0
---	---	---

2.2.14 Define EX-OR and EX-NOR Gates

- **EX-OR (XOR) Gate:** Output is **1** when the inputs are different.
 - **EX-NOR (XNOR) Gate:** Output is **1** when the inputs are the same.
-

2.2.15 Explain the Working of EX-OR and EX-NOR Gates with Truth Tables

EX-OR (XOR)

$$A \oplus B = Y$$

0 0 0

0 1 1

1 0 1

1 1 0

EX-NOR (XNOR)

$$A \odot B = Y$$

0 0 1

0 1 0

1 0 0

1 1 1

2.2.16 Implement AND, OR, NOT Gates Using NAND Gates Only

- **NOT Gate:**
 $A' = A \text{ NAND } A$
- **AND Gate:**
 $A \cdot B = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$
- **OR Gate:**
 $A + B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$

2.2.17 Implement AND, OR, NOT Gates Using NOR Gates Only

- **NOT Gate:**
 $A' = A \text{ NOR } A$
- **OR Gate:**
 $A + B = (A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$
- **AND Gate:**
 $A \cdot B = (A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$

2.3 Basic Combinational Circuits

2.3.1 Define the Half Adder

A **Half Adder** is a combinational circuit that adds **two 1-bit binary numbers** and produces **Sum** and **Carry** outputs.

2.3.2 Explain the Function of Half Adder

- Inputs: **A, B**
- Outputs: **Sum (S)** and **Carry (C)**
- **Sum** = $A \oplus B$
- **Carry** = $A \cdot B$

Truth Table

A B S C

0 0 0 0

0 1 1 0

1 0 1 0

1 1 0 1

2.3.3 Draw Half-Adder Circuit Using EX-OR and AND Gate

- **Sum** = $A \oplus B$ (XOR Gate)
 - **Carry** = $A \cdot B$ (AND Gate)
-

2.3.4 Draw a Half-Adder Using Only NAND Gates or Only NOR Gates

A Half Adder can be implemented using **only NAND gates** or **only NOR gates**, since both are **universal gates**.

2.3.5 Define the Full Adder

A **Full Adder** is a combinational circuit that adds **three 1-bit binary inputs** (A, B, Carry-in) and produces **Sum** and **Carry-out**.

2.3.6 Explain the Function of Full Adder

- Inputs: **A, B, Cin**
- Outputs: **Sum (S)** and **Carry (Cout)**

Equations:

- $S = A \oplus B \oplus C_{in}$
 - $C_{out} = AB + BC_{in} + AC_{in}$
-

2.3.7 Construct Full Adder Using Two Half Adders and an OR Gate

- First Half Adder adds **A and B**.
 - Second Half Adder adds **Sum** and **Cin**.
 - OR gate combines both carry outputs to produce **Cout**.
-

2.3.8 Define the Parallel Adder

A **Parallel Adder** is a digital circuit that adds **two binary numbers simultaneously** using multiple Full Adders.

2.3.9 Explain the Function of Parallel Adder

A Parallel Adder performs addition of all bits **at the same time**, making it faster than a serial adder.

2.3.10 Draw and Explain 4-bit Parallel Adder Using Full Adders

A **4-bit Parallel Adder** consists of **four Full Adders** connected in series. Each Full Adder adds one bit and passes the carry to the next stage.

2.3.11 Draw and Explain 4-bit Parallel Adder / 2's Complement Subtractor

A **4-bit Adder/Subtractor** uses four Full Adders and XOR gates. It performs:

- **Addition** when control input = **0**
 - **Subtraction (2's complement)** when control input = **1**
-

2.3.12 Explain the Working of a Serial Adder

A **Serial Adder** adds binary numbers **one bit at a time** using:

- One Full Adder
- Shift Registers
- Flip-Flop for Carry

It is slower but requires less hardware.

2.3.13 Advantages and Disadvantages of Serial Adder

Advantages:

- Simple design
- Low cost
- Requires less hardware

Disadvantages:

- Slow operation
 - Takes more time for multi-bit addition
-

2.3.14 Advantages and Disadvantages of Parallel Adder

Advantages:

- High speed
- Adds all bits simultaneously
- Suitable for large computations

Disadvantages:

- More hardware required
 - Higher cost
 - Greater power consumption
-

2.3.15 Difference Between Serial Adder and Parallel Adder

Serial Adder	Parallel Adder
Adds one bit at a time	Adds all bits simultaneously
Slow	Fast
Less hardware	More hardware
Low cost	High cost

2.3.16 Explain the Operation of a Digital Comparator Circuit for Two 4-bit Words

A **4-bit Digital Comparator** compares two 4-bit binary numbers (**A₃A₂A₁A₀** and **B₃B₂B₁B₀**) and produces one of three outputs:

- **A > B**
- **A = B**
- **A < B**

Applications:

- Digital computers
- Microprocessors
- Arithmetic Logic Units (ALU)
- Digital control systems

3.0 Flip-Flops

A Flip-Flop is a bistable digital circuit that stores one bit (0 or 1) of data. It is the basic memory element used in sequential circuits.

Types of Flip-Flops

1. SR (Set-Reset) Flip-Flop
2. JK Flip-Flop
3. D (Data) Flip-Flop
4. T (Toggle) Flip-Flop

Features

- Stores one binary bit.
- Has two stable states (0 and 1).
- Changes state only when a control signal (clock) is applied.
- Used in sequential logic circuits.

Applications

- Registers
- Counters
- Memory devices
- Shift registers
- Digital clocks
- Control circuits

3.1 List the Details of Different Logic Families

A logic family is a group of digital integrated circuits (ICs) that use the same electronic technology and have similar electrical characteristics.

Different Logic Families:

1. RTL (Resistor-Transistor Logic)

2. DTL (Diode-Transistor Logic)
3. TTL (Transistor-Transistor Logic)
4. ECL (Emitter-Coupled Logic)
5. MOS (Metal-Oxide Semiconductor)
6. CMOS (Complementary Metal-Oxide Semiconductor)

Features of Logic Families

- RTL: Simple design, low speed.
- DTL: Better noise immunity than RTL.
- TTL: High speed, widely used in digital circuits.
- ECL: Fastest logic family, high power consumption.
- MOS: Low power consumption, high integration.
- CMOS: Very low power consumption, high speed, and widely used in modern digital systems.

Applications:

- Digital computers
- Microprocessors
- Memory circuits
- Embedded systems
- Digital electronic devices

3.2 Define Positive and Negative Logic Levels

- Positive Logic: Logic 1 = High voltage, Logic 0 = Low voltage.
- Negative Logic: Logic 1 = Low voltage, Logic 0 = High voltage.

3.3 Define Flip-Flop

A Flip-Flop is a bistable sequential circuit that stores one bit of binary data (0 or 1).

3.4 Basic Principle of Operation of a Flip-Flop

A Flip-Flop has two stable states and changes its output only when a trigger or clock pulse is applied. It stores one bit until the next trigger.

3.5 Define Latch

A Latch is a memory device that stores one bit of data. Its output changes whenever the input changes while it is enabled.

3.6 Explain the Working of a NAND Latch

A NAND Latch is made using two cross-coupled NAND gates.

Truth Table

S	R	Q(next)	Operation
1	1	No Change	Hold
0	1	1	Set
1	0	0	Reset
0	0	Invalid	Not Allowed

Timing Diagram: During Set, Q becomes 1. During Reset, Q becomes 0. Otherwise, the previous state is retained.

3.7 Explain the Working of a NOR Latch

A NOR Latch is made using two cross-coupled NOR gates.

Truth Table

S	R	Q(next)	Operation
0	0	No Change	Hold
1	0	1	Set

0 1 0 Reset

1 1 Invalid Not Allowed

Timing Diagram: Q changes according to the Set and Reset inputs and retains its previous state when both inputs are 0.

3.8 Difference Between Latch and Flip-Flop

Latch Flip-Flop

Level-triggered Edge-triggered

No clock required Clock required

Faster More reliable

Simpler circuit More complex circuit

3.9 Define Triggering

Triggering is the process of changing the state of a Flip-Flop using a clock pulse.

3.9.1 Types of Triggering

1. Positive Edge Triggering
2. Negative Edge Triggering
3. Positive Level Triggering
4. Negative Level Triggering

3.10 Explain Edge Triggering

- Positive Edge Triggering: Output changes only during the rising edge ($0 \rightarrow 1$) of the clock.
- Negative Edge Triggering: Output changes only during the falling edge ($1 \rightarrow 0$) of the clock.

3.11 Explain Level Triggering

- Positive Level Triggering: Flip-Flop responds while the clock is at HIGH level.
- Negative Level Triggering: Flip-Flop responds while the clock is at LOW level.

3.12 Working of RS Flip-Flop

Inputs: S (Set), R (Reset)

Truth Table

S	R	Q(next)
0	0	No Change
0	1	0
1	0	1
1	1	Invalid

Working: Used to set, reset, or store one bit of data.

3.13 Working of RS-T (Clocked RS) Flip-Flop

The RS-T Flip-Flop is an RS Flip-Flop controlled by a clock input. It changes its state only when the clock is active.

Truth Table

CLK	S	R	Q(next)
0	X	X	No Change
1	0	0	No Change
1	0	1	0
1	1	0	1
1	1	1	Invalid

3.14 Working of D Flip-Flop

Truth Table

CLK	D	Q(next)
↑	0	0
↑	1	1

Working: The output follows the input D at the active clock edge.

3.15 Working of JK Flip-Flop

Truth Table

J	K	Q(next)
0	0	No Change
0	1	0
1	0	1
1	1	Toggle

Working: Eliminates the invalid state of the RS Flip-Flop.

3.16 Working of T Flip-Flop

Truth Table

T	Q(next)
0	No Change
1	Toggle

Working: Toggles its output whenever T = 1.

3.17 Difference Between Synchronous and Asynchronous Inputs

Synchronous Asynchronous

Depends on clock Independent of clock

Changes only with clock Changes immediately

More controlled Faster response

3.18 Need for Master-Slave Flip-Flop

A Master-Slave Flip-Flop is used to eliminate the race-around condition in JK Flip-Flops and ensure reliable operation.

3.19 Working of Master-Slave Flip-Flop

A Master-Slave Flip-Flop consists of two Flip-Flops:

- Master operates during the HIGH clock level.
- Slave operates during the LOW clock level.

This ensures that the output changes only once per clock pulse, preventing race-around problems.

Truth Table (JK Master-Slave)

J	K	Q(next)
0	0	No Change
0	1	0
1	0	1
1	1	Toggle

4.0 Counters and Registers

Counters and **Registers** are important **sequential digital circuits** used in digital electronics.

Counter

A **Counter** is a sequential circuit that counts the number of clock pulses. It changes its output state for every clock pulse received.

Applications:

- Digital clocks
- Event counting
- Frequency counters
- Timers

Register

A **Register** is a group of flip-flops used to **store binary data temporarily**. Each flip-flop stores **one bit** of information.

Applications:

- Data storage
- Data transfer
- Shift operations
- Microprocessors and digital systems

Difference Between Counter and Register

Counter	Register
Counts clock pulses	Stores binary data
Output changes with each clock pulse	Holds data until changed
Used for counting	Used for data storage and transfer

4.1 Counters

4.1.1 Define Counter

A **Counter** is a sequential digital circuit that counts the number of clock pulses and produces a binary output.

4.1.2 List the Types of Counters

1. Asynchronous (Ripple) Counter
 2. Synchronous Counter
 3. Up Counter
 4. Down Counter
 5. Up/Down Counter
 6. Decade (MOD-10) Counter
 7. Ring Counter
 8. Programmable Counter
-

4.1.3 Define Synchronous Counter and Asynchronous Counter

Synchronous Counter:

A counter in which **all flip-flops receive the clock pulse simultaneously**.

Asynchronous (Ripple) Counter:

A counter in which **only the first flip-flop receives the clock pulse**, and the remaining flip-flops are triggered by the previous flip-flop.

4.1.4 Difference Between Asynchronous and Synchronous Counters

Asynchronous Counter	Synchronous Counter
Clock applied only to first flip-flop	Clock applied to all flip-flops
Slow operation	Fast operation
Ripple delay exists	No ripple delay

Asynchronous Counter

Simple design

Synchronous Counter

More complex design

4.1.5 MOD-8 Ripple Counter

A **MOD-8 Ripple Counter** uses **3 flip-flops** and counts from **0 to 7** (000 to 111).

Truth Table

Decimal Output (Q2 Q1 Q0)

0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Working: Each flip-flop is triggered by the output of the previous flip-flop. After **111**, it returns to **000**.

4.1.6 MOD-16 Ripple Counter

A **MOD-16 Ripple Counter** uses **4 flip-flops** and counts from **0 to 15** (0000 to 1111).

Working: It counts 16 clock pulses and then resets to **0000**.

4.1.7 MOD-10 (Decade) Asynchronous Counter

A **Decade Counter** counts from **0 to 9** (0000 to 1001). After reaching **9**, it resets to **0**.

Applications: Digital clocks and frequency counters.

4.1.8 MOD-8 Synchronous Counter

A **MOD-8 Synchronous Counter** uses **3 flip-flops** with a common clock. All flip-flops change state simultaneously.

Count Sequence:

000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000

4.1.9 MOD-16 Synchronous Counter

A **MOD-16 Synchronous Counter** uses **4 flip-flops** connected to the same clock. It counts from **0 to 15** with high speed.

4.1.10 Drawbacks of Ripple Counters

1. Slow operation due to ripple delay.
 2. Accumulated propagation delay.
 3. Not suitable for high-speed circuits.
 4. Temporary false outputs may occur.
-

4.1.11 Advantages of Synchronous Counters

1. High-speed operation.
 2. No ripple delay.
 3. Better accuracy.
 4. Suitable for high-frequency applications.
-

4.1.12 Up/Down Counter

An **Up/Down Counter** can count in **ascending (Up)** or **descending (Down)** order depending on the control input.

Up Count:

000 → 001 → 010 → ... → 111

Down Count:

111 → 110 → 101 → ... → 000

4.1.13 Need for Programmable Counter

A **Programmable Counter** is used when counting must stop or reset at a user-selected value instead of a fixed value.

4.1.14 Programmable Counter

A **Programmable Counter** allows the user to preset the counting limit (MOD value). It is widely used in timers, digital clocks, and frequency dividers.

4.1.15 4-bit Ring Counter

A **4-bit Ring Counter** consists of **4 flip-flops** connected in a ring. Only one flip-flop contains **1**, while the others contain **0**.

Sequence:

1000 → 0100 → 0010 → 0001 → 1000

Applications: Sequence generation and timing circuits.

4.1.16 Applications of Counters

- Digital clocks
- Frequency counters
- Timers

- Event counting
- Traffic light controllers
- Digital measuring instruments
- Microprocessors and digital systems

R.2 Registers

4.2.1 Define Register

A **Register** is a group of flip-flops used to **store binary data temporarily**. Each flip-flop stores **one bit** of information.

4.2.2 State the Need of Register

Registers are used to:

1. Store binary data temporarily.
 2. Transfer data within digital systems.
 3. Perform shift operations.
 4. Support arithmetic and logical operations.
 5. Interface with processors and memory.
-

4.2.3 List the Methods of Data Transfer in Register

1. Serial Data Transfer
 2. Parallel Data Transfer
-

4.2.4 List the Types of Registers

1. Serial-In Serial-Out (SISO)
2. Serial-In Parallel-Out (SIPO)

3. Parallel-In Serial-Out (PISO)
 4. Parallel-In Parallel-Out (PIPO)
 5. Shift Left Register
 6. Shift Right Register
 7. Universal Shift Register
-

4.2.5 Define Serial-In Serial-Out (SISO) Register

A **SISO Register** accepts data **serially (one bit at a time)** and also gives the output **serially**.

4.2.6 Define Serial-In Parallel-Out (SIPO) Register

A **SIPO Register** accepts data **serially** and provides the output **simultaneously (parallel)**.

4.2.7 Define Parallel-In Serial-Out (PISO) Register

A **PISO Register** accepts data **in parallel** and transmits it **serially**, one bit at a time.

4.2.8 Define Parallel-In Parallel-Out (PIPO) Register

A **PIPO Register** accepts **parallel data** and provides **parallel output** simultaneously.

4.2.9 Explain the Working of Serial-In Serial-Out (SISO) Register

A **SISO Register** consists of flip-flops connected in series. Data enters one bit at a time with each clock pulse and shifts through the register. The output is obtained serially from the last flip-flop.

Application: Serial data communication.

4.2.10 Explain the Working of Serial-In Parallel-Out (SIPO) Register

In a **SIPO Register**, data enters serially. After sufficient clock pulses, all bits become available simultaneously at the outputs.

Application: Serial-to-parallel data conversion.

4.2.11 Explain the Working of Shift Left Register

In a **Shift Left Register**, data moves **one bit toward the left** on every clock pulse. The leftmost bit is shifted out, and a new bit enters from the right.

Application: Multiplication by 2 and data shifting.

4.2.12 Explain the Working of Shift Right Register

In a **Shift Right Register**, data moves **one bit toward the right** on every clock pulse. The rightmost bit is shifted out, and a new bit enters from the left.

Application: Division by 2 and serial data transmission.

4.2.13 Explain the Working of Universal Shift Register

A **Universal Shift Register** can perform **four operations**:

1. Shift Left
2. Shift Right
3. Parallel Load
4. Hold (No Change)

It is called **universal** because it supports all basic shift register operations.

4.2.14 Draw and Explain the Use of Shift Register as Memory

A **Shift Register** can be used as a **temporary memory** because it stores binary data in flip-flops. Data remains stored until it is shifted out or overwritten by new data.

Applications:

- Temporary data storage
- Data buffering
- Delay circuits
- Serial communication
- Digital signal processing

5.0 Additional Combinational Circuits

Additional Combinational Circuits are digital circuits whose outputs depend only on the **present input values**. They do **not have memory** or feedback. These circuits are used for **data selection, data routing, coding, decoding, and arithmetic operations**.

Examples of Additional Combinational Circuits

1. Multiplexer (MUX)
2. Demultiplexer (DEMUX)
3. Encoder
4. Decoder
5. Priority Encoder
6. Magnitude Comparator

Applications

- Data transmission and communication
- Digital computers and microprocessors
- Memory addressing
- Arithmetic Logic Units (ALU)
- Control systems
- Digital signal processing

5.1 Define Data Selector and State Its Importance

A **Data Selector** is a digital circuit that selects **one input from many inputs** and transfers it to a **single output** based on the select lines. It is also called a **Multiplexer (MUX)**.

Importance:

- Selects one input from multiple inputs.
 - Reduces the number of data lines.
 - Used in data transmission and communication.
 - Saves hardware and wiring.
 - Widely used in digital systems and microprocessors.
-

5.2 Define Data Distributor and State Its Importance

A **Data Distributor** is a digital circuit that receives **one input** and distributes it to **one of many outputs** based on the select lines. It is also called a **Demultiplexer (DEMUX)**.

Importance:

- Distributes one input to multiple outputs.
- Used in communication systems.
- Reduces hardware complexity.
- Used in memory addressing and data routing.
- Widely used in digital control systems.

5.3 Define the Terms Multiplexer and Demultiplexer

Multiplexer (MUX)

A **Multiplexer (MUX)** is a **data selector** that selects **one input from many inputs** and sends it to a **single output** based on the select lines.

Demultiplexer (DEMUX)

A **Demultiplexer (DEMUX)** is a **data distributor** that receives **one input** and distributes it to **one of many outputs** according to the select lines.

5.4 Draw and Explain the Operation of a Multiplexer with Truth Table

A **Multiplexer (MUX)** is a combinational circuit that selects **one of many input lines** and transfers it to a **single output** using **select lines**.

Example: 4-to-1 Multiplexer

Inputs: I_0, I_1, I_2, I_3

Select Lines: S_1, S_0

Output: Y

Truth Table

$S_1 S_0$ Output (Y)

0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Working

- When **$S_1 S_0 = 00$** , I_0 is selected.
- When **$S_1 S_0 = 01$** , I_1 is selected.
- When **$S_1 S_0 = 10$** , I_2 is selected.
- When **$S_1 S_0 = 11$** , I_3 is selected.

Applications:

- Data selection
- Data routing
- Communication systems
- Digital computers and microprocessors

5.5 Draw and Explain the Operation of Demultiplexer with Truth Table

A **Demultiplexer (DEMUX)** is a combinational circuit that accepts **one input** and routes it to **one of many outputs** based on the **select lines**.

Example: 1-to-4 Demultiplexer

Input: D

Select Lines: S_1, S_0

Outputs: Y_0, Y_1, Y_2, Y_3

Truth Table

$S_1 S_0$ Active Output

0 0 $Y_0 = D$

0 1 $Y_1 = D$

1 0 $Y_2 = D$

1 1 $Y_3 = D$

Working

- When $S_1 S_0 = 00$, the input **D** is sent to Y_0 .
- When $S_1 S_0 = 01$, the input **D** is sent to Y_1 .
- When $S_1 S_0 = 10$, the input **D** is sent to Y_2 .
- When $S_1 S_0 = 11$, the input **D** is sent to Y_3 .

Note: In the exam, draw the **standard 1-to-4 DEMUX logic symbol** with one input (**D**), two select lines (S_1, S_0), and four outputs (Y_0 – Y_3).

5.6 List the Applications of Multiplexers

1. Data selection.
2. Data routing in digital systems.
3. Communication systems.
4. Time-division multiplexing (TDM).
5. Microprocessors and microcontrollers.
6. Memory addressing.
7. Digital signal processing.
8. Arithmetic Logic Units (ALUs).

5.7 List the Applications of Demultiplexers

A **Demultiplexer (DEMUX)** is used in the following applications:

1. Data distribution.
2. Communication systems.

3. Memory address decoding.
4. Data routing.
5. Digital control systems.
6. Serial-to-parallel data conversion.
7. Computer and microprocessor systems.
8. Signal switching circuits.

5.8 Define the Terms Encoder and Decoder

Encoder

An **Encoder** is a combinational circuit that converts **2^n input lines into n output lines**. It converts multiple input signals into a binary code.

Example: An **8-to-3 Encoder** converts **8 input lines** into **3-bit binary output**.

Decoder

A **Decoder** is a combinational circuit that converts **n input lines into 2^n output lines**. It converts a binary code into one of several output lines.

Example: A **3-to-8 Decoder** converts **3-bit binary input** into **8 output lines**.

5.9 Draw and Explain the Operation of a 4-to-10 Line Decoder with Truth Table

A **4-to-10 Line Decoder** is a combinational circuit that converts a **4-bit Binary Coded Decimal (BCD) input** into **10 output lines**. Only **one output is HIGH (1)** for each valid decimal input (0–9).

Inputs: A, B, C, D

Outputs: Y_0 to Y_9

Truth Table

A	B	C	D	Active Output
0	0	0	0	Y_0

A B C D Active Output

0001 Y_1

0010 Y_2

0011 Y_3

0100 Y_4

0101 Y_5

0110 Y_6

0111 Y_7

1000 Y_8

1001 Y_9

Working:

- The decoder accepts a **4-bit input**.
- According to the input value, **only one output line becomes HIGH**, while all others remain LOW.
- Inputs **1010 to 1111** are invalid for a BCD decoder.

Applications:

- Memory address decoding
- Display systems
- Digital control circuits

Note: In the exam, draw the **standard 4-to-10 Decoder logic symbol** showing four inputs (**A, B, C, D**) and ten outputs (**Y_0 – Y_9**).

5.10 Draw and Explain the Operation of an 8-to-3 Encoder with Truth Table

An **8-to-3 Encoder** is a combinational circuit that converts **8 input lines** into a **3-bit binary output**.

Inputs: D_0 to D_7

Outputs: A, B, C

Truth Table

Active Input A B C

D_0	000
D_1	001
D_2	010
D_3	011
D_4	100
D_5	101
D_6	110
D_7	111

Working

- Only **one input** should be HIGH at a time.
- The encoder converts the active input into its **3-bit binary code**.
- For example:
 - If $D_3 = 1$, the output is **011**.
 - If $D_6 = 1$, the output is **110**.

Applications:

- Keyboard encoding
- Data compression
- Digital communication systems
- Microprocessor interfaces

Note: In the exam, draw the **standard 8-to-3 Encoder logic symbol** with eight inputs (D_0 – D_7) and three outputs (A, B, C).

5.11 List the Applications of Decoders

A **Decoder** is used in the following applications:

1. Memory address decoding.
 2. Seven-segment display driving.
 3. Digital clocks and calculators.
 4. Data routing and distribution.
 5. Microprocessor and microcontroller systems.
 6. Code conversion.
 7. Communication systems.
 8. Control circuits.
-

5.12 List the Applications of Encoders

An **Encoder** is used in the following applications:

1. Keyboard encoding.
2. Data compression.
3. Digital communication systems.
4. Priority interrupt systems.
5. Microprocessor interfaces.
6. Code conversion.
7. Data transmission.
8. Digital control systems.

5.13 Define Programmable Logic Devices (PLDs)

A **Programmable Logic Device (PLD)** is a digital integrated circuit (IC) whose logic functions can be **programmed by the user** to perform different digital operations.

5.14 List the Types of Programmable Logic Devices (PLDs)

The main types of Programmable Logic Devices are:

1. **PROM (Programmable Read-Only Memory)**

2. **PAL (Programmable Array Logic)**
 3. **PLA (Programmable Logic Array)**
 4. **GAL (Generic Array Logic)**
 5. **CPLD (Complex Programmable Logic Device)**
 6. **FPGA (Field Programmable Gate Array)**
-

5.15 Draw and Explain the Programmable Logic Array (PLA)

A **Programmable Logic Array (PLA)** is a programmable logic device that has **both programmable AND gates and programmable OR gates**. It is used to implement any Boolean function.

Working

- The **programmable AND array** generates the required product (AND) terms.
- The **programmable OR array** combines these product terms to produce the desired output.
- Since both arrays are programmable, a PLA is more flexible than a PAL.

Advantages

1. High flexibility.
2. Can implement complex Boolean functions.
3. Reduces the number of logic gates.
4. Easy to modify the logic design.

Applications

- Digital control systems.
- Microprocessors.
- Embedded systems.
- Industrial automation.
- Digital communication systems.